Segment Trees

Point updates, range queries, and walking down the tree — all in O(log n)

Learning goals

- Explain why prefix sums fail once the array **changes**
- Understand the segment tree **shape**: nodes = dyadic segments
- Implement **recursive** and **iterative (bottom-up)** segment trees
- Prove a range query touches only **O(log n)** nodes
- Design **custom nodes** (max subarray sum, count + min, ...)
- **Walk the tree**: first index with value ≥ x, k-th one, in O(log n)

Motivating problem — CSES 1649

Dynamic Range Minimum Queries: n, q ≤ 2·10^5

- `1 k u` — set `a[k] = u`
- `2 a b` — print `min(a[a..b])`

Approach	Update	Query
Plain array	O(1)	O(n)
Recompute prefix structure	O(n)	O(1)
Segment tree	**O(log n)**	**O(log n)**

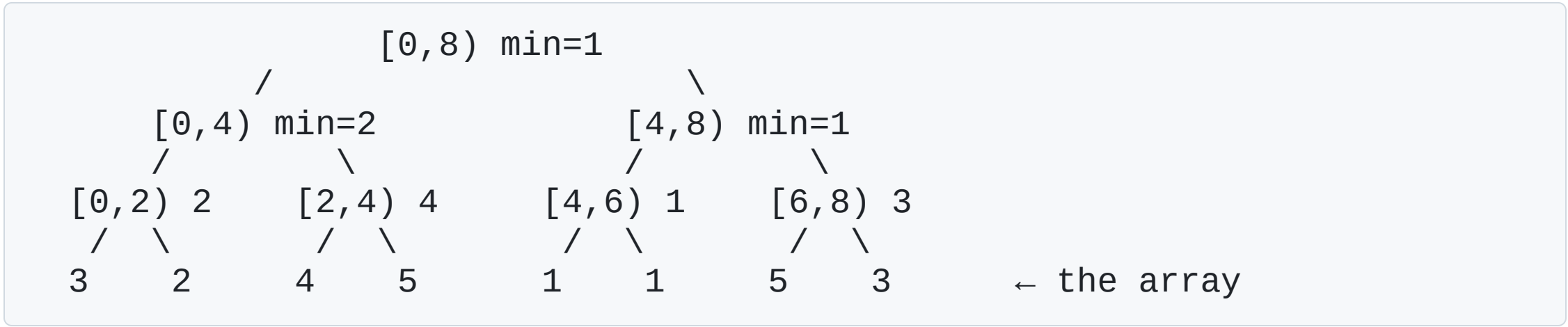
The idea

Precompute answers for a set of segments such that

1. Every query range = union of **few** precomputed segments
2. Every position lies in **few** precomputed segments

Halving does both: root = `[0, n)`, children = left half, right half, ...

The shape (n = 8)



- Height ⌈log2 n⌉, ≈ 2n nodes
- Each node stores the **combine** of its two children

Storing it in an array

- Node `x` has children `2x` and `2x+1`, root = 1
- Recursive version: allocate **4n** (safe for any n)
- Iterative version: exactly **2n**, leaves at `n .. 2n−1`

```
index:  1    2    3    4    5    6    7    8 .. 15
node : [0,8)[0,4)[4,8)[0,2)[2,4)[4,6)[6,8) leaves
```

Recursive build & update

```
void build(int x, int l, int r) {
    if (l == r) { t[x] = a[l]; return; }
    int m = (l + r) / 2;
    build(2*x, l, m); build(2*x+1, m+1, r);
    t[x] = min(t[2*x], t[2*x+1]);
}
void update(int x, int l, int r, int pos, int v) {
    if (l == r) { t[x] = v; return; }
    int m = (l + r) / 2;
    if (pos <= m) update(2*x, l, m, pos, v);
    else          update(2*x+1, m+1, r, pos, v);
    t[x] = min(t[2*x], t[2*x+1]);
}
```

Update = one root-to-leaf path: O(log n).

Recursive range query

```
int query(int x, int l, int r, int ql, int qr) {
    if (qr < l || r < ql) return INT_MAX;          // disjoint
    if (ql <= l && r <= qr) return t[x];           // fully inside
    int m = (l + r) / 2;                           // partial: split
    return min(query(2*x, l, m, ql, qr),
               query(2*x+1, m+1, r, ql, qr));
}
```

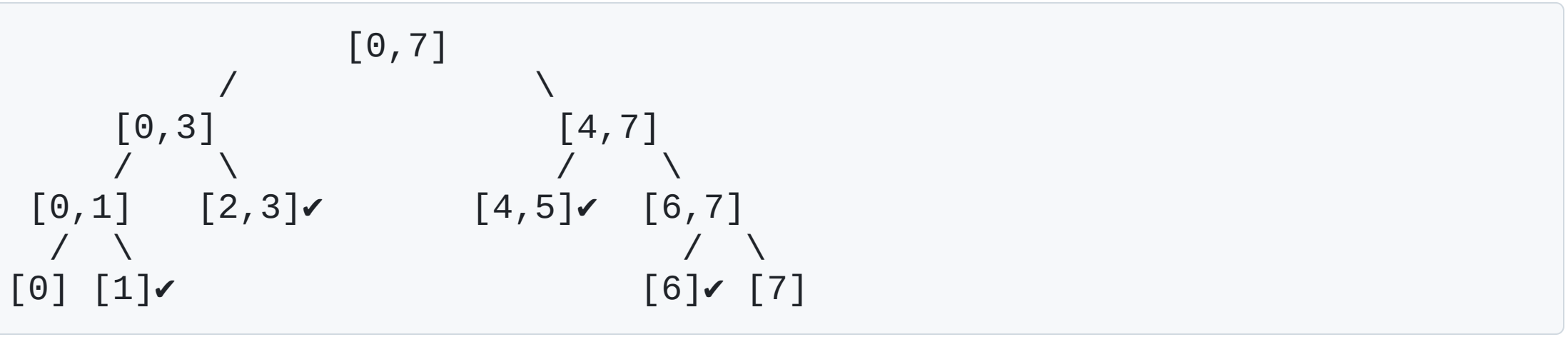
Three cases: **disjoint**, **covered**, **partial**. `INT_MAX` is the **identity** of `min`.

Why O(log n)?

Claim: on each level, at most **2** nodes are *partial* (recursed into further).

- A partial node contains an endpoint `ql` or `qr` of the query
- Each endpoint lies in exactly one node per level
- ⇒ ≤ 2 partial nodes per level, ≤ 4 visited nodes per level
- ⇒ ≤ 4·⌈log2 n⌉ nodes visited in total

Query example: min of a[1..6]



Answer = min(t[1], t[2,3], t[4,5], t[6]) — 4 covered nodes.

Iterative (bottom-up) segment tree

```
// leaves t[n..2n-1]; t[i] = min(t[2i], t[2i+1])
void update(int p, int v) {
    for (t[p += n] = v; p > 1; p >>= 1)
        t[p >> 1] = min(t[p], t[p ^ 1]);
}
int query(int l, int r) {              // [l, r)  half-open!
    int res = INT_MAX;
    for (l += n, r += n; l < r; l >>= 1, r >>= 1) {
        if (l & 1) res = min(res, t[l++]);
        if (r & 1) res = min(res, t[--r]);
    }
    return res;
}
```

Short, fast, works for **any n** if the operation is commutative.

Reading the bottom-up loop

- `l` odd ⇒ `l` is a **right child**: its parent sticks out to the left → take `t[l]`, move right
- `r` odd ⇒ `t[r-1]` is a **left child** inside the range → take it
- Then go one level up on both sides
- Non-commutative operation (e.g. matrix product, string hash)? Keep **two** accumulators `resL`, `resR` and combine `resL · resR` at the end

Choosing an operation

Works whenever `combine` is **associative** with an **identity**:

Query	combine	identity
sum	`a + b`	0
min / max	`min` / `max`	$+\infty$ / $-\infty$
gcd	`gcd(a,b)`	0
xor	`a ^ b`	0
min and its count	merge pairs	$(\infty, 0)$

Custom nodes — max subarray sum

CSES 1190: point updates, after each print the **max subarray sum**.

```
struct Node { long long sum, pref, suf, best; };
Node combine(const Node& L, const Node& R) {
    return {L.sum + R.sum,
            max(L.pref, L.sum + R.pref),
            max(R.suf,  R.sum + L.suf),
            max({L.best, R.best, L.suf + R.pref})};
}
```

Ask: *what does a parent need to know that can be computed from its children alone?*

Walking the tree

CSES 1143 *Hotel Queries*: for each group of size r, find the **first** hotel with ≥ r free rooms.

- Binary search + range max query: $O(\log^2 n)$
- **Descend** using stored maxima: $O(\log n)$

```
int firstAtLeast(int x, int l, int r, int need) {
    if (mx[x] < need) return -1;
    if (l == r) return l;
    int m = (l + r) / 2;
    if (mx[2*x] >= need) return firstAtLeast(2*x, l, m, need);
    return firstAtLeast(2*x+1, m+1, r, need);
}
```

Walking: k-th one

Array of 0/1, sum tree. Find the position of the **k-th one**:

```
int kth(int x, int l, int r, int k) {    // 1 <= k <= t[x]
    if (l == r) return l;
    int m = (l + r) / 2;
    if (t[2*x] >= k) return kth(2*x, l, m, k);
    return kth(2*x+1, m+1, r, k - t[2*x]);
}
```

- CSES 1749 *List Removals*: delete the k-th remaining element
- Order statistics on values (after compression): "k-th smallest in a multiset"

Walking with a left bound

"First index **≥ L** with `a[i] ≥ x`":

- Recurse as in a range query restricted to `[L, n-1]`
- In each fully covered node: if `max < x` skip it, else descend inside it and stop
- Still O(log n): only one covered node is ever descended into

Segment tree over values

Index the tree by **value** instead of position:

- `cnt[v]` = how many times value v is present (compress first!)
- Count of values in `[lo, hi]` = range sum
- k-th smallest = k-th one walk
- CSES 1144 *Salary Queries*: change a salary, count salaries in [a, b]

Complexity

Operation	Time
build	O(n)
point update	O(log n)
range query	O(log n)
descent / walk	O(log n)
memory	2n (iterative) / 4n (recursive)

n = q = 2·10^5 → about 4·10^6 node visits. Comfortable.

Recursive vs iterative

	Recursive	Iterative
Code length	longer	very short
Speed	~2–3× slower	fast
Walks / descents	natural	awkward
Lazy propagation (Week 25)	natural	possible, tricky
Non-commutative ops	natural	two accumulators

Learn both; use the iterative one for plain query + update.

Common pitfalls

- Array size `2n` for the **recursive** version → out of bounds; use `4n`
- Mixing **inclusive** `[l, r]` and **half-open** `[l, r)` conventions
- Wrong identity (e.g. `0` for min)
- Forgetting to recompute the parent after an update
- `int` overflow in sums
- Iterative tree + **non-commutative** combine with one accumulator
- Rebuilding the whole tree per update (O(n))

Practice problems

- CSES 1648 *Dynamic Range Sum Queries*
- CSES 1649 *Dynamic Range Minimum Queries*
- CSES 1143 *Hotel Queries* (walk)
- CSES 1749 *List Removals* (k-th one)
- CSES 1144 *Salary Queries* (value tree)
- CSES 1190 *Subarray Sum Queries* (custom node)
- Codeforces 339D *Xenia and Bit Operations*
- Codeforces 474F *Ant colony* (gcd + count of min)
- Codeforces 380C *Sereja and Brackets* (custom node)

Summary

- Segment tree = precomputed answers on **dyadic segments**
- Any **associative** combine with an identity
- Update: one path, O(log n). Query: ≤ 4 nodes per level, O(log n)
- Bottom-up version: 2n memory, ~10 lines
- **Walk** the tree to answer "first / k-th" questions in O(log n)
- Next: Fenwick trees & sparse tables (24), lazy propagation (25)